

Case note - WimpForth fsave abort

WimpForth: an fsave that aborts the machine

A case note from the RISC OS Pi 4 emulator. WimpForth metacompiles its own kernel on the emulator without trouble, but saving the finished application image kills the desktop with "abort on data transfer" — in a large WimpSlot, and never in a small one.

The case at a glance

Status	Unsolved at the operating-system level. A workaround is in place and validated.
Filed	15 September 2026
Reproduces	Every time WimpForth saves its image while running in a large WimpSlot
Machine	RISC OS 5.30 on QEMU's emulated Raspberry Pi 4, booting from HostFS 2.03
Affects	WimpForth's whole-image save, and plausibly any program that saves a large, partly untouched memory range with <code>OS_File 10</code>

The short version

WimpForth saves its running image by handing `OS_File 10` the whole range from `&8000` to the top of its dictionary. In a large WimpSlot much of that range has never been touched, so its pages are not yet mapped. The save is copied in SVC mode, where touching an unmapped page is a data abort rather than a page fault the kernel can quietly serve, and the machine goes down.

Reading one byte of every page from user mode before the save — which makes the demand pager map them — lets the same save complete in the same slot. That is the workaround. What is still not known is exactly which copy loop faults, and why the range reaches past mapped memory in the first place.

Where the source is

WimpForth's upstream is [alban-read/forth](#). The emulator, its HostFS 2.03 module and the `vmchannel` device it talks to are in [albanread/RISCOSQEMUA72](#), branch `riscos-pi4`. Details of the private test farm the case was worked on are shown as `<redacted>`.

Chapters

Chapter	What it covers
1. The case as filed	what works, what dies, the save path, and the first theory
2. A second opinion	the right HostFS, a path that can abort, a lost digit, and a plan
3. The evidence	registers at the fault, the RAMFS control, the slot-size control, the workaround
4. A second bug on the way	a one-character delimiter, and why guest sources stay Latin-1
5. Conclusions	what this says about WimpForth, and what is still not established
6. Status and open items	the order the remaining questions would be picked up in

How to read the diagrams

The diagrams colour each box's outline by who owns that code.

Outline	Meaning
purple	the application: WimpForth
grey	RISC OS itself
navy	a filing system: HostFS or RAMFS
teal	the host, or the emulator's device
red	where the fault happens

1. The case as filed

What works

WimpForth runs as `FWIN32`, an Absolute file of 159,904 bytes, launched by `WimpTask` from `HostFS:$.WimpForth`. The upstream is [alban-read/forth](#); the case was worked on a local fork at `<redacted>`.

Test	Result
The interpreter: <code>1 2 + .</code>	<code>3 ok</code>
Loading source with <code>FLOAD</code>	<code>works</code>
Full metacompilation: <code>FLOAD META32</code>	a 27,020-byte kernel image
<code>TARGET-SAVE kernel32</code>	written to HostFS, byte-identical to the shipped <code>kernel32</code> , every time

The saved kernel's MD5 is `aefefe1dcf1fa8c123f01ad5f4de4266`, the same as upstream's.

What dies

Building the application is a different story. `FLOAD MAKEWIN`, or simply `fsave TESTSAVE`, compiles around twenty source files and prints `Extensions Loaded, 3054 words in dictionary`. Then comes the `FSAVE` step, which dumps the whole running image to a file through `OS_File 10`, and the machine dies:

```
Application has gone wrong: abort on data transfer at &2006E08
```

In a TaskWindow the same step does not raise an error box: it simply wedges, with no `ok` prompt. After the error box the desktop is unusable, and the virtual machine has to be restarted.

The save path

The save goes through two short pieces of WimpForth, both upstream and unmodified.

The kernel's OS_File word, and save-file

```
code OS_File                \ r0 <- TOS, then r1..r5 from stack
  mov r0, tos
  ldmfd sp !, { r1, r2, r3, r4, r5, tos }
  swi x " OS_File"
next c;
: save-file ( ad len filename -- )
  1+ >r bounds 0 &ff8 r> 10 OS_File ;
```

"fsave in UTILS

```
: "fsave ( a1 n1 -- )
  fsave-buf place fsave-buf count over >r + 0 swap c!
  32768 here over - r> 1-      \ start=&8000, end=here
  save-file ;
```

"fsave saves everything from 32768 — &8000, the classic base of application space — up to here, the top of the dictionary, as an Absolute file (type &FF8).

The first theory

The theory filed with the case was that the hardcoded 32768 image base was wrong for a task running higher in memory. Chapter 2 is where that was re-examined.

2. A second opinion

A second look at the case came back with six points. They are summarised here, lightly edited.

The HostFS that was read was not the one running

The farm's ROM carries HostFS 2.03. The HostFS source read for the case was the retired 1.x module, and the two save files differently.

On 2.x, whole-file save is retired. `OS_File 10` never reaches `FSEntry_File 0`. Instead FileSwitch builds the save out of Open, a run of buffered PutBytes calls with a 1,024-byte buffer, and Close. Each PutBytes goes over the `C_FS_PUTBYTES` command, which passes the guest's logical address and walks the MMU page by page.

The mismatch the case had noted between an 8,192 and a 4,032 `VMCH_MAX_ARG` was an artefact of comparing 1.x with 2.x: a red herring.

HostFS 2.x has a path that can abort the guest

The emulator's device cannot fault a guest page in: it can only read pages that are already mapped. So on a short transfer HostFS first runs [touch_pages](#), which reads one byte of every page, in the guest's SVC mode, to make sure they are.

Its own comment says what would happen with a wild range — it aborts there — and names the fix: validate the range with `OS_ValidateAddress` first.

That fitted the symptoms. The 27 KB kernel image had only just been written, so it was mapped and never needed the fallback. The `FSAVE` range covered the whole task, much of it never touched.

The crash address had lost a digit

`OS_ConvertHex8` always prints eight digits, so `&2006E08` — seven — lost one when it was copied down. Which digit matters, because the RISC OS 5.3 memory map puts the two candidates in very different places.

Address range	What lives there
<code>&8000</code> to <code>&1FFFFFFF</code>	application space
from <code>&20000000</code>	the RMA — module code and data
from <code>&30000000</code>	the system heap

So `&02006E08` is ordinary application space, which needs a slot of at least 32 MB, while `&20006E08` is in the RMA, among module code. And "Application has gone wrong" is the Wimp's wording, not WimpForth's.

The first probe should not be discarded

An earlier reading gave `coldstart = &2008000`. That is exactly `&8000` plus 32 MB — the top of a 32 MB slot — and `here = &2006E08` sits just below it, so the two readings agree with each other.

Later readings of `A3800` and `2F0Ax` fit a 640 KB slot instead. Both can be true, of different launches: `WimpTask` from the job runner, against `!RUN` with its own `WimpSlot` line. The advice was to measure the slot at the moment of the crash.

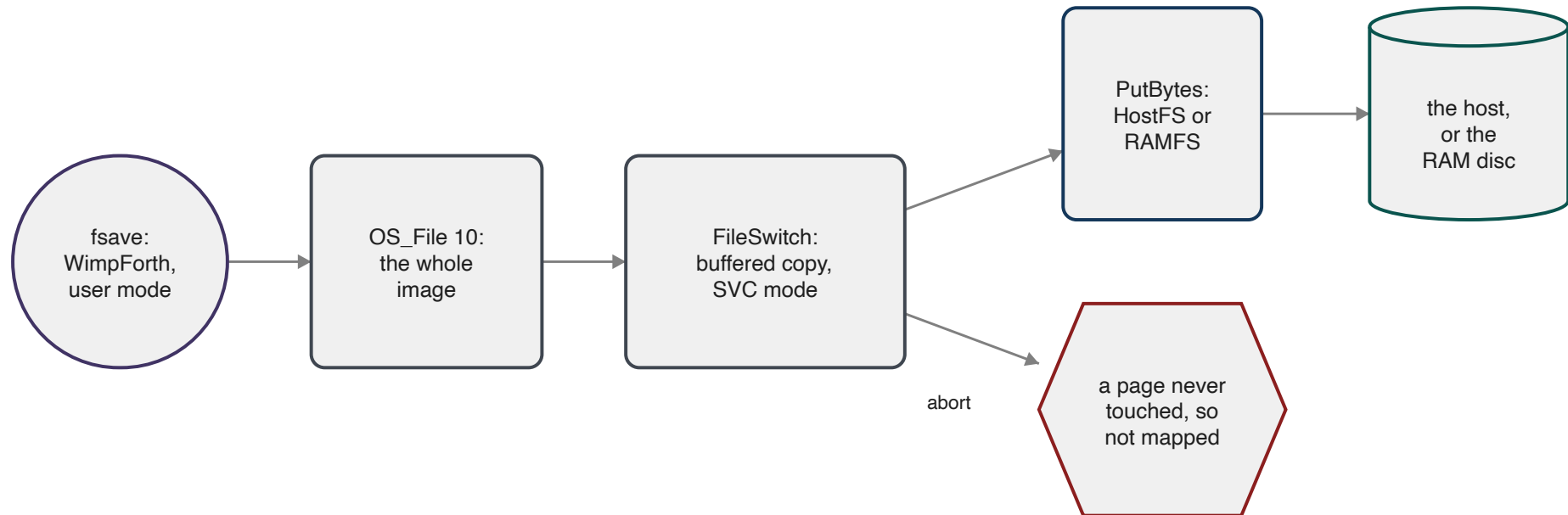
A plan, and a fix worth making regardless

The order to run things in:

Step	Why
1. *ShowRegs and *Where, redirected to a file	the mode says user or SVC; *Where names the module
2. The same save to RAMFS	is this HostFS at all?
3. The failing save with VMCH_TRACE on	where the device loses the mapping
4. Log R4, R5 and the slot size from inside "fsave"	the real range, at the real moment
5. Only then, bisect size and range	

Whatever the outcome, PutBytes and GetBytes should call `OS_ValidateAddress` before touching pages, and return an error instead of aborting.

The save path, and where it faults



3. The evidence

The research followed the order from chapter 2.

Step 1: the registers at the fault

*ShowRegs and *Where were redirected to a file on HostFS, so the dump could be read exactly rather than by OCR from a screenshot.

Register	Value
R0, R1	&001DD000, &1A8
R2, R4, R8–R11	&00B5E8B5
R3, R5	&00B2E4B3, &0018D69C
R12, R13	&FFFF12B0, &FA207FA8
R14, R15	&FC038210, &FC0382B8
Mode and PSR	SVC32, &20000113

The dump was stored at &20003110, and *Where placed the PC at offset &00017FB4 in the module UtilityModule.

What the registers say

The faulting context is in SVC32, inside the kernel. It is not WimpForth's own user-mode code, and the PC is not in a filing-system module. FileSwitch's buffered `OS_File 10` copy loop is the natural candidate.

The buffer registers point at `&00B5E8B5` and `&0018D69C` — deep inside a 32 MB slot and far beyond the 160 KB image, which is the untouched region being walked when the abort fired. The RMA address in the Wimp's box and the kernel PC in the dump are consistent with the dump having been captured in the abort handler.

Step 2: the RAMFS control

A 64 MB RAM disc was made with `ChangeDynamicArea -RamFsSize 64M`, and the same save sent to it with `fsave RAM::RamDisc0.$.TESTSAVE`.

It **aborted identically**, and the job runner then hung on the modal error box. A `*ShowRegs` after the box was dismissed printed a dump identical to the HostFS one: the same stored-context address, `&20003110`, and the same registers.

What the RAMFS control does and does not prove

Two caveats are kept on record. The RAMFS error box was read by OCR prompted with the address it was expected to show, and the identical dump may be a stale stored context rather than a fresh one. But the runner hanging after the RAMFS save is hard evidence that the RAMFS save killed the desktop too.

So the fault is **not HostFS-specific**. The mechanism the second opinion described — a save spanning pages that are not yet mapped, faulting during an SVC-mode copy — survives. On this evidence the faulting copy is FileSwitch's own buffered save, upstream of any filing system.

The slot-size control

The cheapest discriminator was run next, ahead of its turn.

Launch	WimpSlot	fsave TESTSAVE
through the application's own !RUN	-max 2m -min 2m	succeeds: 159,916 bytes on the host, runner alive
through WimpTask	the default, large	aborts

The crash depends on the size of the slot, exactly as the second opinion's reading of the two probes predicted. In a big slot, the range handed to `OS_File` — or the layout `WimpForth` derives from its RAM limit — extends over pages the task has never touched.

The workaround: touch the pages first

If the problem is pages that have never been touched, touching them first from user mode should fix it, because a user-mode read of an unmapped page is an ordinary page fault the demand pager serves.

Reading one byte of every page before the save

```
: touchit 32768 here over - bounds do i c@ drop 4096 +loop ;  
touchit fsave TESTSAVE2      \ in the SAME big-slot configuration
```

`TESTSAVE2`, 160,000 bytes, saved cleanly, and the machine survived. Once every page in the range has been read from Forth, the SVC-mode copy has nothing left to fault on. The fix is now folded into UTILS as `"fsave` calling a `touch-range` word (fork commit <redacted>).

4. A second bug on the way

A rebuild that still failed

The first full rebuild with the workaround in place still failed, visibly and live on the farm's display:

```
R stack underflow at DEBUG line 533
```

One character, one byte

An earlier housekeeping change in the fork had transliterated the delimiter on the PASM and DEBUG banner comments from `comment ö` to `comment oe`.

`comment` takes **one character** as the delimiter it skips to. With `oe` it took `o` — and `o` occurs inside the commented block, in `#patchinto`. So the skip ended in the middle of the block, and half of a dead definition executed.

What the tree learned

Sources that are loaded into the guest are kept in Latin-1 precisely because constructs like this depend on single bytes. Strings shown at run time may be plain ASCII, but a delimiter has to keep its original byte.

The fix restored the Latin-1 `ö` delimiters (fork commit `<redacted>`).

5. Conclusions

What this says about WimpForth

WimpForth's save model assumes the flat machine of its era: the image at `&8000`, everything between `&8000` and `here` already touched, and an application slot that fits snugly. Under a modern, large WimpSlot, `here` — or the layout the kernel derives from `OS_GetEnv`'s RAM limit — leaves the saved range spanning pages that have never been touched, and `OS_File 10`'s buffered SVC-mode copy is not prepared for that.

This is a genuine portability defect in WimpForth, not a misconfigured farm. The same binary works in a 2 MB slot and dies in a big one.

Why kernel builds always worked

The metacompiler is unaffected. `TARGET-SAVE` saves a buffer it has only just written, so every page of it is mapped. That is why rebuilding the kernel always worked while rebuilding the application died.

The bootstrap now completes

With two small fixes in the WimpForth tree — touching the range before `save-file`, and restoring the Latin-1 delimiters — the full documented bootstrap completes on the farm.

Stage	Result
Metacompile <code>kernel32</code>	byte-identical to upstream
<code>fload</code> <code>makewin</code> on the bare kernel	loads
Save FWIN	159,984 bytes written
Run the new FWIN	boots and computes: <code>1 2 + .</code> gives <code>3 ok</code>

The freshly built `FWIN32` is committed to the fork.

What is not yet established

These are why the case stays open.

Question	What would settle it
Which kernel instruction is at <code>UtilityModule +0x17FB4</code> , and whose copy loop it is	a ROM with symbols, or *Where before the handler runs
Whether the box's <code>&20006E08</code> is the real faulting PC, or an artefact of the handler	the RAMFS error box, captured without a primed prompt
Why the saved range reaches past mapped memory in a big slot	a read of WimpForth's RAM-limit layout

On the last of these: WimpForth's cold start puts `rp` at `OS_GetEnv`'s RAM limit less 16 KB, and the end of the dictionary reportedly lands at the top of a 32 MB slot. Whether that is what carries the save range past the mapped pages is exactly what has not yet been read.

6. Status and open items

Status

Unsolved at the operating-system level. A workaround is in place and validated.

The workaround lives in the WimpForth tree, as UTILS's `"fsave` calling `touch-range` (fork commit `<redacted>`), and every rebuild from now on exercises it.

Open items

In the order they would be picked up.

#	Item
1	Capture the RAMFS error box without a primed prompt — a fresh screenshot, no expected address — and a <code>*ShowRegs</code> straight after a RAMFS-only abort, to settle RMA against kernel PC.
2	Log the real R4, R5 and slot size to a file from inside <code>"fsave</code> , just before the SWI: step 4 of the plan, not yet run.
3	Run a HostFS save under <u>VMCH_TRACE</u> : a <code>translate failed at va=</code> line would show exactly where the range stops being mapped.

Open items, continued

#	Item
4	Identify UtilityModule +017FB4 in a ROM with symbols.
5	Harden HostFS regardless: call OS_ValidateAddress in PutBytes and GetBytes before touching pages, so a dead virtual machine becomes an error the client can report.
6	Write up the WimpForth fix for upstream. Touching the range first is generic: it applies to any Forth that saves a whole image through OS_File 10 on RISC OS 5 in a large slot.

The environment

Host	a Mac Pro running macOS
Emulator	RISCOSQEMUA72 's qemu-system-aarch64 -M raspi4b -cpu cortex-a72,aarch64=off
ROM	stock RISC OS 5.30 with HostFS 2.03 spliced in (<redacted>); no network
HostFS share	<redacted>, file types carried as ,xxx suffixes
Driven by	a private test farm, over QMP: run, type, screenshot (<redacted>)

Launching WimpForth

Launch	Slot	Outcome
Dir HostFS:\$.!WimpForth, then WimpTask FWIN32	large	aborts on save
through !RUN, which sets WimpSlot 2m	2 MB	saves

The source and the evidence

What	Where
WimpForth, upstream	alban-read/forth
The WimpForth fork the case was worked on	<redacted>
HostFS 2.03 module source, touch_pages at line 651	riscos-pi4/hostfs/dde/c/hostfs
The vmchannel device	hw/misc/vmchannel.c
Register dumps after the HostFS and RAMFS aborts; the saved TESTSAVE, TESTSAVE2 and FWIN; screenshots of both abort boxes and the new FWIN running	<redacted>